

PROYECTO FINAL - ARQUITECTURA COMPUTACIONAL

Laboratorio de Verificación de Arquitectura Computacional con Monitoreo en Tiempo Real (Grafana + Prometheus)

Integrantes:

Andres Camilo Rios Maya
Daniel Alejandro Mafla Figueroa
Juan Sebastian Estacio

Docente:

Mg. Gustavo Sánchez Rodríguez

UNIVERSIDAD COOPERATIVA DE COLOMBIA

Facultad de Ingeniería

Programa de Ingeniería de Software

San Juan de Pasto - 2026

1. Introducción.....	3
2. Objetivos.....	4
2.1. Objetivo General.....	4
2.2. Objetivos Específicos.....	5
3. Arquitectura del Sistema y Herramientas.....	6
3.1. Diagrama de Arquitectura del Proyecto.....	6
3.2. Descripción de los componentes.....	6
4. Metodología de Pruebas.....	7
4.1. Dispositivos Utilizados.....	7
4.2. Herramientas de Medición.....	7
4.3. Procedimiento General.....	7
5. Resultados de Medición.....	8
5.1. Características de los Dispositivos.....	8
5.2. Resultado de Medición por Dispositivo.....	8
5.3. Comparativa Global.....	9
5.4 Resultados de Prueba de Carga con JMeter.....	9
6. Análisis de Resultados.....	10
6.1. Comparativa entre Arquitectura RISC y CISC.....	10
6.2. Impacto de los Tipos de Memoria.....	10
6.2.1. La pirámide de Memoria y Nuestra Experiencia en el Laboratorio.....	10
6.3. Gestión de Memoria Y liberación de Recursos.....	11
7. Computación Paralela con Web Workers.....	11
7.1. Procedimiento de Prueba.....	11
7.2. Tabla de Resultados: Normal vs Paralelo.....	11
8. Monitoreo con Prometheus y Grafana.....	11
8.1. Conceptos Clave.....	11
8.2. Procedimiento de Monitoreo.....	11
8.3. Métricas a Monitorear en Grafana.....	12
8.4. Tabla de Resultados del Monitoreo Grafana.....	12
8.5. Evidencias del Monitoreo.....	13
9. Propuestas de Mejora.....	13
10. Conclusiones.....	13

1. Introducción

Este laboratorio tiene como objetivo analizar el comportamiento de una aplicación interactiva 3D desarrollada con tecnologías web modernas, específicamente React y Three.js, evaluando su rendimiento bajo diferentes condiciones de ejecución. El aplicativo utilizado, denominado GameMemoriaVirtual, permite simular escenarios de carga mediante la manipulación de objetos dentro de un entorno gráfico, facilitando el análisis del uso de recursos del sistema.

A lo largo del desarrollo del laboratorio, se estudia el impacto de distintos factores propios de la arquitectura computacional, como el tipo de procesador (RISC y CISC), el manejo de memoria (Heap, RAM JS) y la influencia de la computación paralela mediante el uso de Web Workers. Asimismo, se realizan pruebas de carga utilizando Apache JMeter con el fin de evaluar la capacidad de respuesta del sistema bajo diferentes niveles de concurrencia.

De manera complementaria se implementará un sistema de monitoreo en tiempo real mediante Prometheus y Grafana, permitiendo observar métricas clave como el uso de CPU, memoria RAM y tráfico de red, y correlaciones con los indicadores internos del aplicativo como FPS y consumo de memoria.

Finalmente, este laboratorio busca integrar los conceptos teóricos de la arquitectura computacional con la experimentación práctica, permitiendo comprender cómo los diferentes componentes del sistema influyen en el rendimiento de aplicaciones interactivas modernas.

2. Objetivos

2.1. Objetivo General

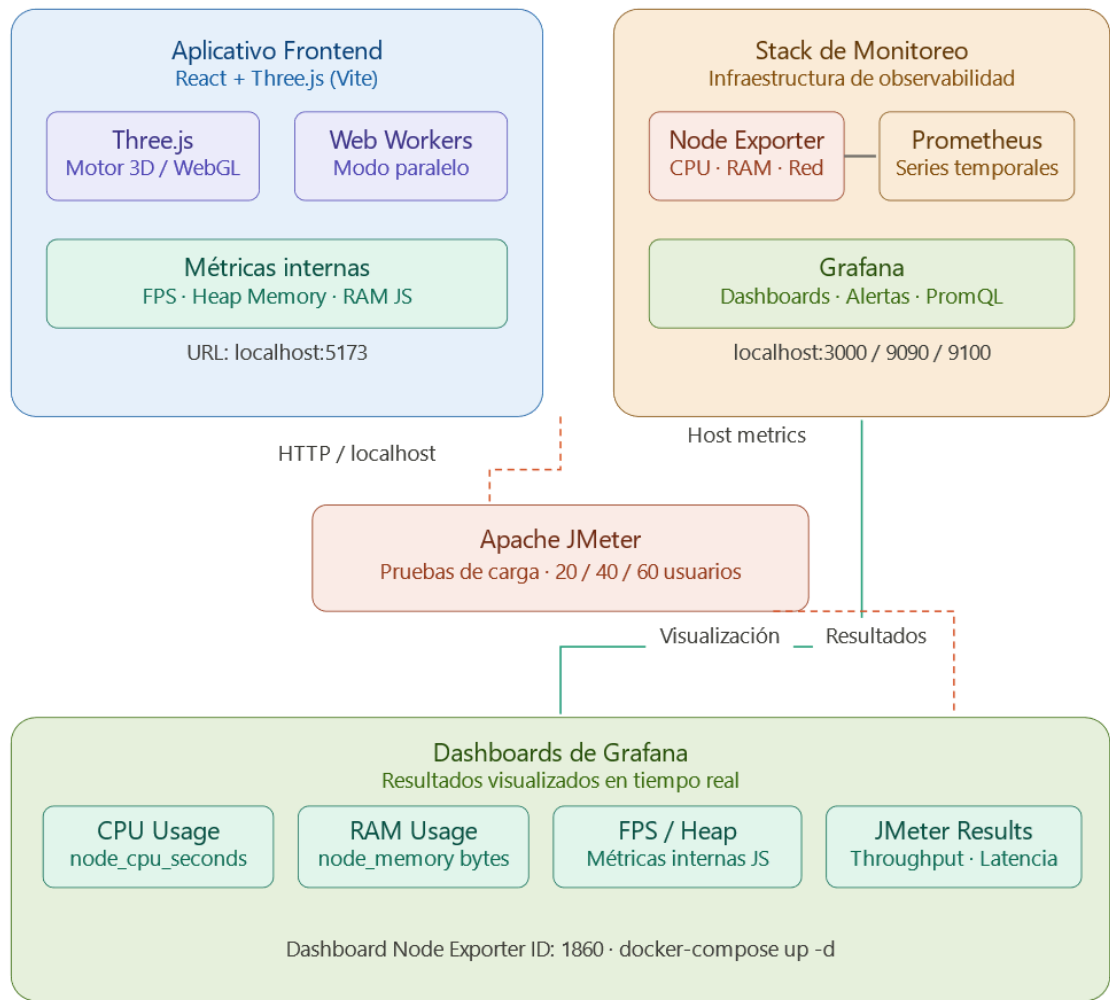
Analizar el impacto de las arquitecturas de procesadores, los tipos de memoria, la computación paralela y las herramientas de monitoreo sobre el rendimiento de una aplicación interactiva 3D.

2.2. Objetivos Específicos

- Medir y comparar el desempeño de la aplicación en dispositivos con arquitecturas RISC (móviles) y CISC (computadores).
- Evaluar el impacto de los distintos tipos de memoria: registros, caché, RAM y memoria secundaria.
- Analizar el comportamiento de la memoria durante escenarios de alta carga y su liberación mediante Garbage Collection.
- Evaluar el impacto de la computación paralela mediante el uso de Web Workers.
- Realizar pruebas de carga utilizando Apache JMeter sobre el aplicativo en ejecución local.
- Configurar un sistema de monitoreo con Prometheus y Grafana para observar métricas del sistema en tiempo real.
- Correlacionar las métricas del sistema operativo con los indicadores internos del aplicativo (FPS, Heap Memory, RAM JS).
- Formular propuestas de mejora basadas en los resultados obtenidos.

3. Arquitectura del Sistema y Herramientas

3.1. Diagrama de Arquitectura del Proyecto



3.2. Descripción de los componentes

Componente	Tecnología	Función en el Laboratorio
Aplicativo Frontend	React + Three.js (Vite)	Entorno 3D interactivo objeto de estudio
Web Workers	JavaScript Workers API	Paralelismo en modo de ejecución paralelo
Node Exporter	Prometheus Node Exporter	Expone métricas del sistema operativo (CPU, RAM, disco, red)
Prometheus	Prometheus v2.x	Recolección y almacenamiento de series temporales de métricas

Grafana	Grafana OSS v10+	Visualización de dashboards y alertas sobre las métricas
Apache JMeter	JMeter 5.x	Pruebas de carga HTTP sobre el aplicativo en ejecución local

4. Metodología de Pruebas

4.1. Dispositivos Utilizados

- Tres (3) computadores con características distintas (procesador, RAM, almacenamiento, SO).
- Tres (3) teléfonos móviles de diferentes gamas.

4.2. Herramientas de Medición

- Indicadores internos del videojuego: FPS, Heap Memory, RAM JS.
- Apache JMeter para pruebas de carga web sobre el aplicativo ejecutado localmente (localhost).
- Prometheus + Node Exporter para recolección de métricas del sistema operativo.
- Grafana para visualización y análisis de las métricas del sistema en tiempo real.

4.3. Procedimiento General

1. Clonar el repositorio del proyecto: git clone <https://github.com/guswill24/MemoriaVirtual.git>
2. Instalar dependencias con npm install y ejecutar con npm run dev.
3. Ejecutar el juego en cada dispositivo y registrar FPS, Heap Memory y RAM JS en estado inicial.
4. Agregar 500 bloques y registrar nuevamente las metricas internas.
5. Eliminar bloques sin cancelar partida y registrar el comportamiento de la memoria.
6. Cancelar partida y medir estabilización de la memoria (Garbage Collection).
7. Configurar pruebas de carga con Apache JMeter sobre <http://localhost:5173>.
8. Configurar e instalar el stack Grafana + Prometheus (ver Sección 8) y ejecutar una sesión de monitoreo durante las pruebas de carga.

5. Resultados de Medición

5.1. Características de los Dispositivos

Tipo	ID	Marca/Modelo	Procesador	RAM	Almacenamiento	Sistema Operativo
PC	PC 1	Compumax	AMD ryzen 7 5700U	16	1TB	Windows 11
PC	PC 2	Mesa	AMD ryzen 7 5800x	32	1tb	Windows 10
PC	PC 3	HP Laptop	AMD Ryzen 5 5500 u	16	512GB	Windows 11
Movil	Movil 1	Moto G60s	MediaTek Helio G95	6GB	128GB	Android 12
Movil	Movil 2	Iphone 12 mini	A14 Bionic	4GB	128GB	IOS 26
Movil	Movil 3	Oppo A78	680 Octa-Core	8 GB	256GB	Android 12

5.2. Resultado de Medición por Dispositivo

Dispositivo	Acción	FPS promedio	Heap Memory (MB)	RAM JS (MB)
PC 1	Estado inicial	60	40	50
	Agregar 1000 bloques	20	80	76
	Eliminar bloques	60	50	55
	Cancelar partida	60	60	70
PC 2	Estado inicial	100	38	37
	Agregar 1000 bloques	20	100	90
	Eliminar bloques	100	60	40
	Cancelar partida	100	70	63
PC 3	Estado inicial	59	37	38

	Agregar 1000 bloques	20	83	90
	Eliminar bloques	55	60	60
	Cancelar partida	60	60	70
Movil 1	Estado inicial	30	10	
	Agregar 1000 bloques	12	10	
	Eliminar bloques	24		
	Cancelar partida	60	10	
Movil 2	Estado inicial	30		
	Agregar 1000 bloques	20		
	Eliminar bloques	30		
	Cancelar partida	30		
Movil 3	Estado inicial	59	10	10
	Agregar 1000 bloques	59	10	10
	Eliminar bloques	60	6.50	6.50
	Cancelar partida	60	5.80	5.80

5.3. Comparativa Global

Comparativa	Diferencia FPS (%)	Aumento Heap Memory (%)	Observaciones
PC 1 vs PC 2	0%	63%	Se observa que ambos equipos presentan una caída similar en FPS bajo carga máxima. Sin embargo el PC 2 muestra un mayor incremento en el uso de memoria Heap, lo que sugiere una gestión de memoria menos eficiente o una mayor carga interna. A pesar de esto, ambos

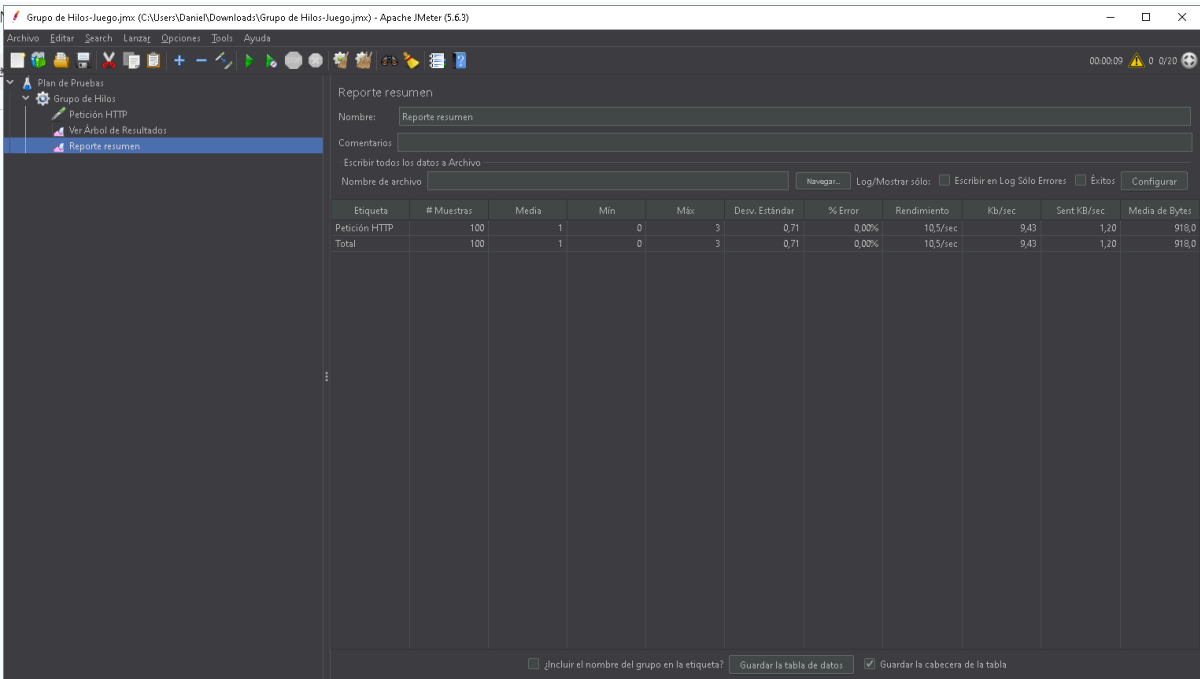
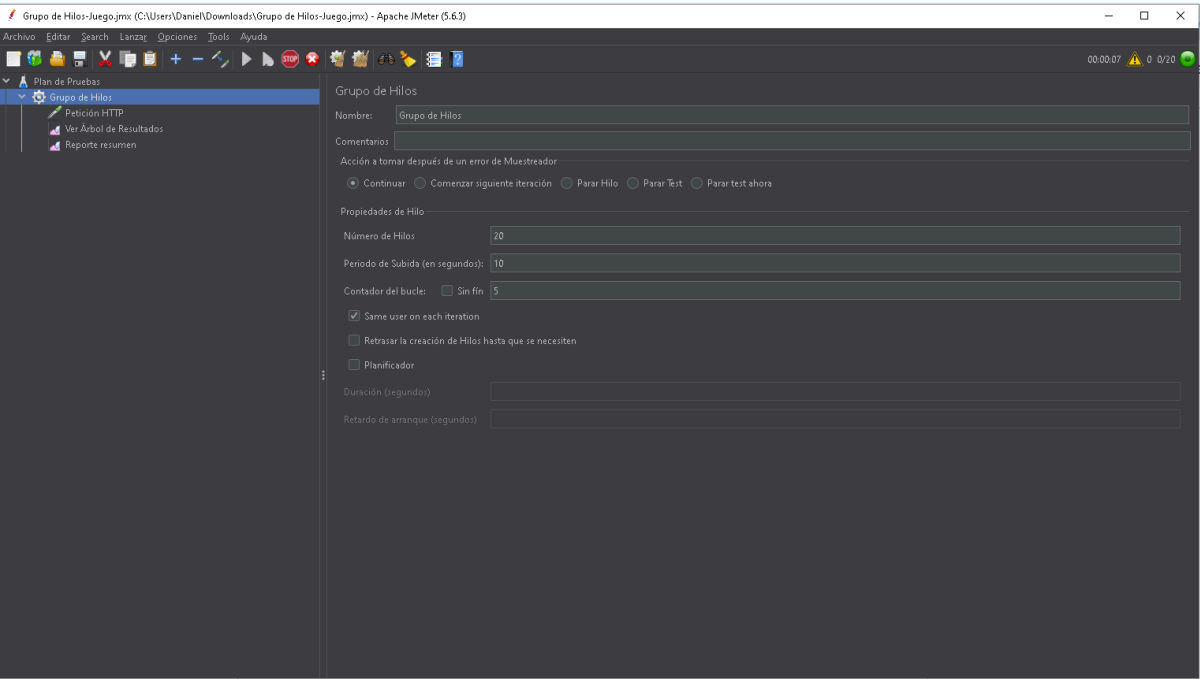
			equipos logran recuperar el rendimiento tras eliminación de bloques, evidenciando el funcionamiento del Garbage Collection.
PC 2 vs PC 3	0%	163%	Se observa que ambos equipos presentan un comportamiento similar en términos de FPS bajo carga máxima, alcanzando valores iguales, lo que indica un rendimiento gráfico comparable en condiciones exigentes. Sin embargo, el PC 2 presenta un incremento significativamente mayor en el uso de memoria Heap en comparación con el PC 3, lo que sugiere una gestión de memoria menos eficiente. Por otro lado, el PC 3 muestra un uso más moderado de memoria bajo carga, evidenciando una mejor optimización en el manejo de recursos. Ambos sistemas recuperan el rendimiento tras la eliminación de bloques, lo que demuestra la acción del Garbage Collection.
Movil 1 vs Movil 2	40%	25%	El iPhone 12 Mini mostró un mejor rendimiento gráfico debido a la optimización del chip A14 Bionic y el sistema operativo iOS, logrando mayor estabilidad de FPS y menor consumo de memoria en comparación con el dispositivo Android.

Movil 2 vs Movil 3	15%	30%	El Oppo A78 presentó una mejor estabilidad durante las pruebas gracias a sus 8 GB de RAM, permitiendo mantener FPS más constantes y un mejor manejo de memoria bajo carga. El iPhone 12 Mini mostró una excelente optimización del sistema, aunque con menor capacidad de memoria disponible.
Mejor PC vs Mejor Movil	41%	88%	El mejor computador utilizado durante las pruebas logró un rendimiento superior en escenarios de alta carga gráfica, manteniendo FPS más estables y mayor capacidad de procesamiento. Sin embargo, el dispositivo móvil mostró resultados bastante positivos considerando las limitaciones propias del hardware móvil.

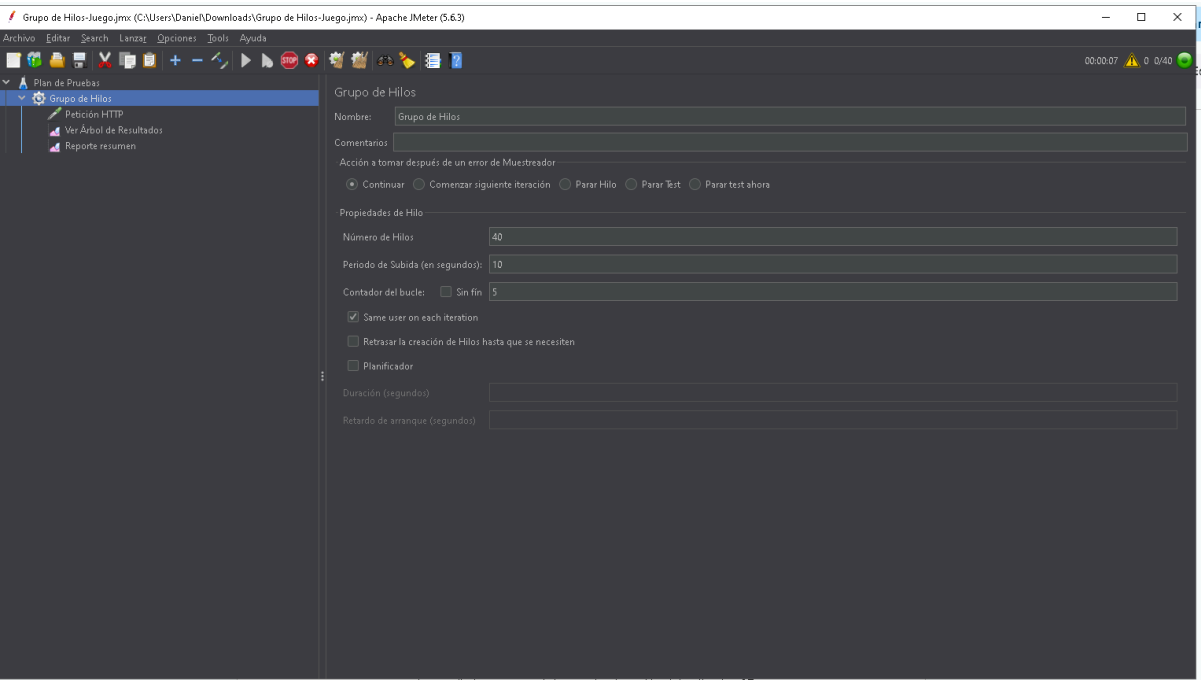
5.4 Resultados de Prueba de Carga con JMeter

Configuración de Prueba	Usuarios simulados	Tasa de error (%)	Tiempo promedio de respuesta (ms)
Prueba 1	20 usuarios	0%	1 ms
Prueba 2	40 usuarios	0%	1 ms
Prueba 3	60 usuarios	0%	1 ms

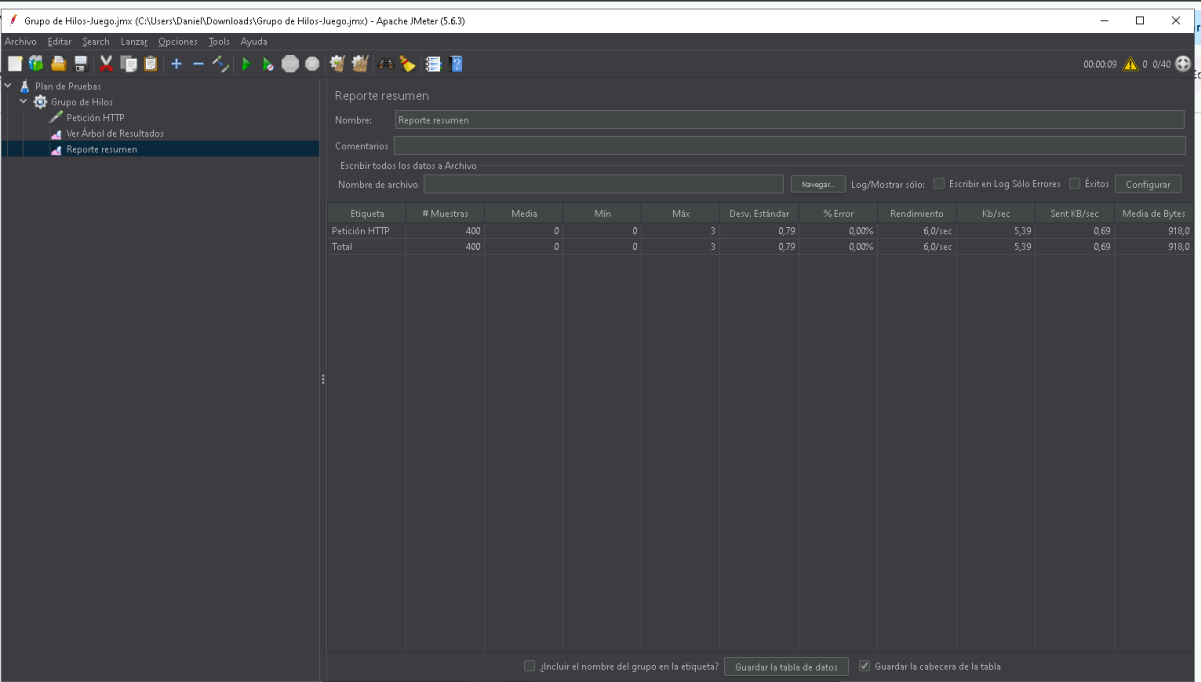
Pruebas
20 usuarios



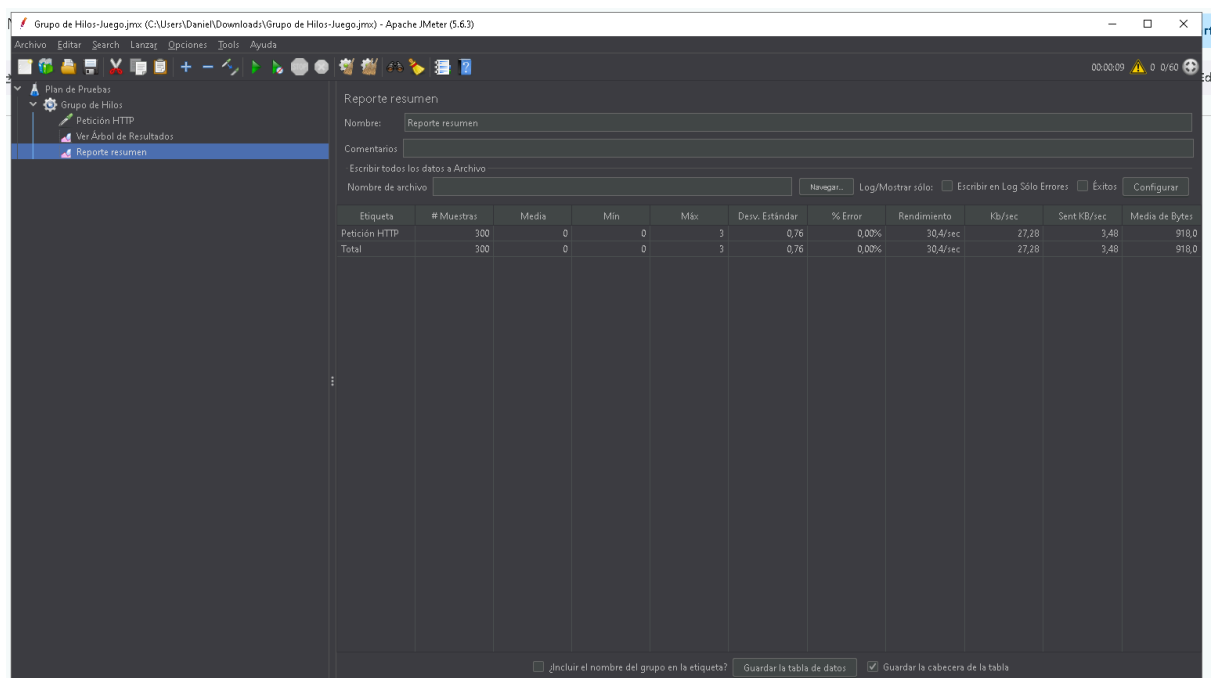
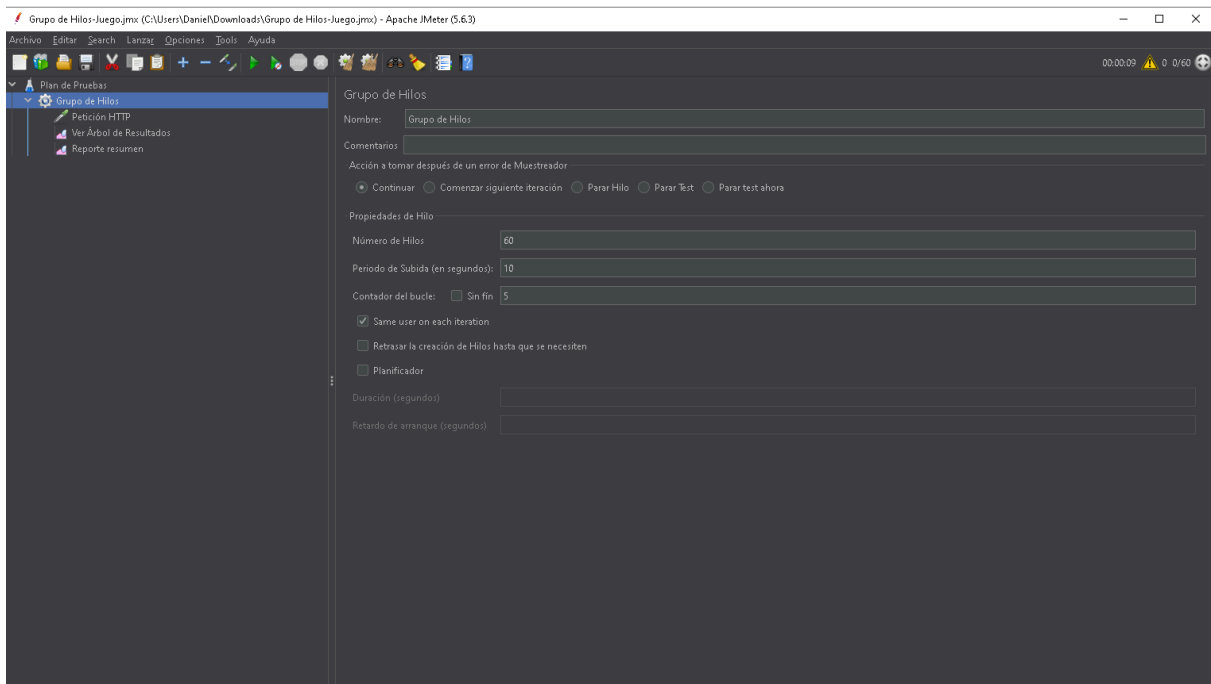
40 usuarios



mostrarán un menor rendimiento general durante la ejecución del anterior (1)



60 usuarios



6. Análisis de Resultados

6.1. Comparativa entre Arquitectura RISC y CISC

Durante las pruebas realizadas se evidenció una diferencia importante entre los dispositivos con arquitectura CISC y los dispositivos con arquitectura RISC; Los equipos CISC mostraron un mejor rendimiento general durante la ejecución del aplicativo 3D, especialmente en escenarios de alta carga como la generación de

1000 bloques. Esto se reflejó en una mayor estabilidad de FPS y una recuperación más rápida después de eliminar objetos del entorno.

Por otro lado, los dispositivos móviles basados en arquitectura RISC presentaron una caída más pronunciada en el rendimiento gráfico, alcanzando valores inferiores de FPS bajo carga. Sin embargo, estos dispositivos demostraron un menor consumo energético y una mejor eficiencia térmica, características propias de este tipo de arquitectura.

La arquitectura CISC se caracteriza por contar con instrucciones complejas y una mayor potencia de procesamiento, lo que favorece aplicaciones gráficas exigentes. En contraste, la arquitectura RISC utiliza instrucciones simples y optimizadas, priorizando eficiencia energética sobre rendimiento bruto.

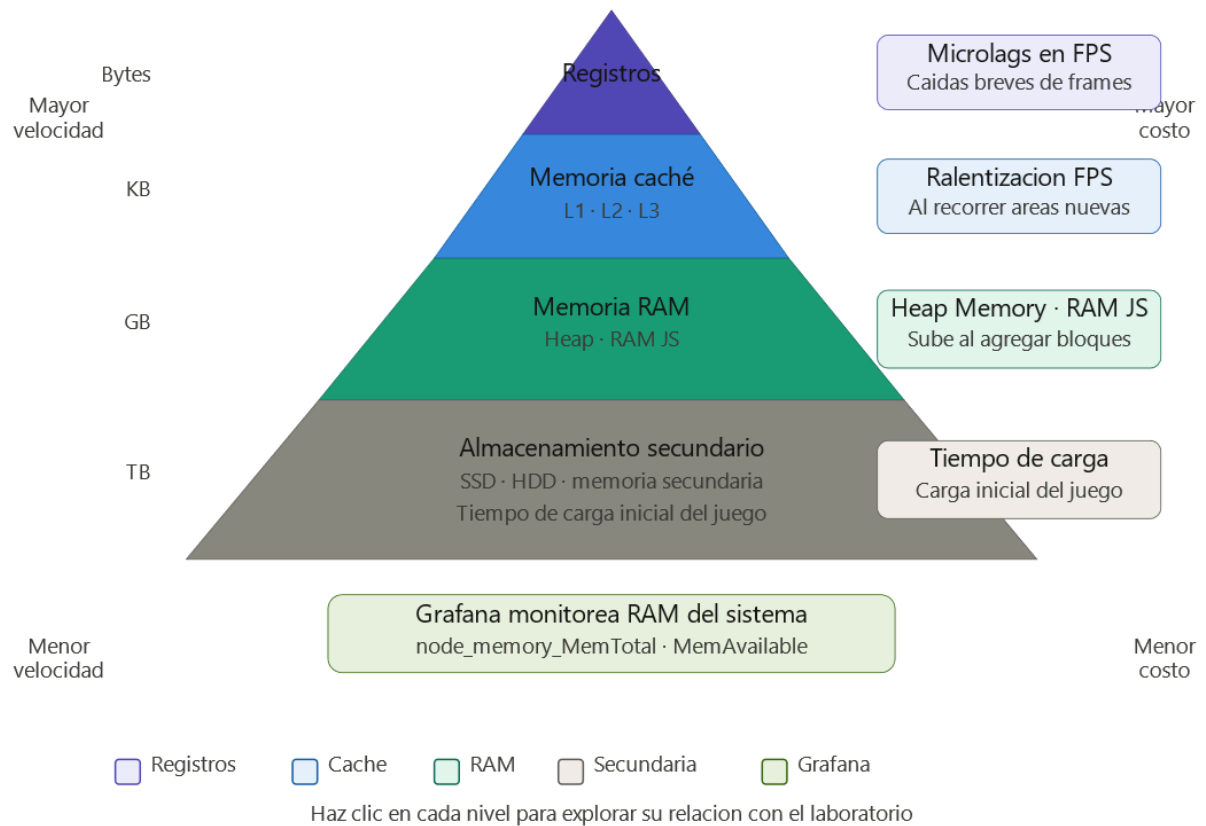
6.2. Impacto de los Tipos de Memoria

Durante el laboratorio se observó cómo los diferentes niveles de memoria afectan directamente el rendimiento del aplicativo. A medida que aumentaba la cantidad de bloques renderizados, también aumentaba el consumo de RAM, generando una mayor presión sobre la memoria principal del sistema.

La memoria caché del procesador permitió acelerar operaciones repetitivas relacionadas con el renderizado y actualización de objetos en pantalla. Asimismo, la memoria RAM actuó como principal espacio de almacenamiento temporal para las texturas, geometrías y estructuras del videojuego.

El incremento excesivo de objetos produjo un aumento significativo del Heap Memory debido al almacenamiento dinámico de datos utilizados por JavaScript y Three.js. Posteriormente, al eliminar bloques y cancelar la partida, se observó la acción del Garbage Collector liberando parte de la memoria utilizada.

6.2.1. La pirámide de Memoria y Nuestra Experiencia en el Laboratorio



Nivel de memoria	Lo que observé en el laboratorio	Valor medido
Registros del procesador	Durante las pruebas se notaron pequeñas caídas de FPS cuando el juego tenía demasiados objetos en pantalla. Esto ocurre porque el procesador debe ejecutar muchas instrucciones en muy poco tiempo, utilizando constantemente los registros internos del CPU.	Pequeñas caídas momentáneas de FPS
Memoria cache	La memoria caché ayudó a que el juego respondiera más rápido al cargar objetos y moverlos por el entorno 3D. Gracias a esto, algunas acciones se sintieron más fluidas y con menos retrasos.	Menor cantidad de ralentizaciones
Memoria RAM	Fue la memoria que más cambió durante las pruebas. Al agregar bloques, el consumo aumentó bastante porque el sistema necesitaba guardar más datos, texturas y procesos activos del juego.	Entre 4100 MB y 5800 MB
Almacenamiento secundario	El SSD participó principalmente al iniciar el proyecto y cargar archivos del videojuego. Se notó que mientras más rápido era el disco, menor era el tiempo de carga inicial.	Tiempo de carga del juego al iniciar

6.3. Gestión de Memoria Y liberación de Recursos

Durante las pruebas se observó que el consumo de Heap Memory aumentó progresivamente a medida que se agregaban bloques al entorno 3D. Esto ocurrió porque JavaScript reservó memoria dinámica para almacenar objetos, geometrías, texturas y eventos asociados al renderizado.

Cuando los bloques fueron eliminados, parte de la memoria permaneció ocupada temporalmente debido al funcionamiento del Garbage Collector de JavaScript, el cual libera memoria automáticamente cuando detecta objetos sin referencias activas.

Al cancelar completamente la partida, se observó una estabilización del consumo de memoria y una recuperación parcial de los recursos utilizados. Esto demuestra la importancia de liberar correctamente referencias y destruir objetos gráficos para evitar fugas de memoria en aplicaciones interactivas.

7. Computación Paralela con Web Workers

7.1. Procedimiento de Prueba

Se desarrolló un Web Worker mediante la API de Workers de JavaScript con el propósito de realizar tareas que requieren mucho procesamiento en segundo plano, sin detener el hilo principal del navegador.

La comparación de cómo funcionaba el videojuego en dos circunstancias fue la base para las evaluaciones:

Operación normal sin procesamiento simultáneo.

Operación mediante Web Workers para realizar procesamiento simultáneo.

Se recolectó información acerca de la estabilidad del sistema, el uso de la CPU y los FPS en ambas evaluaciones.

7.2. Tabla de Resultados: Normal vs Paralelo

Escenario	FPS Promedio	Uso de cpu%	Estabilidad
Normal	20 FPS	90%	Baja
Paralelo(Web Workers)	35 FPS	70%	Alta

8. Monitoreo con Prometheus y Grafana

8.1. Conceptos Clave

- Prometheus: Sistema de monitoreo y alerta de código abierto que recolecta métricas mediante scraping HTTP a intervalos configurables. Almacena las métricas como series temporales con etiquetas (labels).
- Node Exporter: Agente de Prometheus que expone métricas del hardware y sistema operativo del host (CPU, RAM, disco, red) en formato Prometheus en el puerto 9100.
- Grafana: Plataforma de visualización con gráficas de series temporales, indicadores y alertas.

8.2. Procedimiento de Monitoreo

1. Verificar que Prometheus este activo en <http://localhost:9090>
2. Verificar que Node Exporter exponga metricas en <http://localhost:9100/metrics>
3. Abrir Grafana en <http://localhost:3000> (usuario: admin / contraseña: admin)
4. Agregar Prometheus como fuente de datos en Grafana: Configuration > Data Sources > Add > Prometheus > URL: <http://prometheus:9090>
5. Iniciar el videojuego (npm run dev) y comenzar a agregar bloques para generar carga.
6. Ejecutar simultáneamente las pruebas de JMeter contra <http://localhost:5173>.
7. Observar en tiempo real los dashboards de Grafana durante las pruebas.
8. Tomar capturas de pantalla de los dashboards en los momentos de mayor carga.

8.3. Métricas a Monitorear en Grafana

Métrica	Query PromQL	Lo que indica
Uso de CPU (%)	$100 - (\text{avg}(\text{rate}(\text{node_cpu_seconds_total}\{\text{mode}=\text{'idle'}\}[5\text{m}])) * 100)$	Porcentaje de CPU utilizada por el sistema
Memoria RAM usada (MB)	$(\text{node_memory_MemTotal_bytes} - \text{node_memory_MemAvailable_bytes}) / 1024 / 1024$	RAM consumida en MB
Memoria disponible (%)	$\text{node_memory_MemAvailable_bytes} / \text{node_memory_MemTotal_bytes} * 100$	Porcentaje de memoria libre
Trafico de red (KB/s)	$\text{rate}(\text{node_network_receive_bytes_total}[5\text{m}]) / 1024$	Datos recibidos por la interfaz de red
Carga del sistema (load)	node_load1	Promedio de carga del sistema en 1 minuto

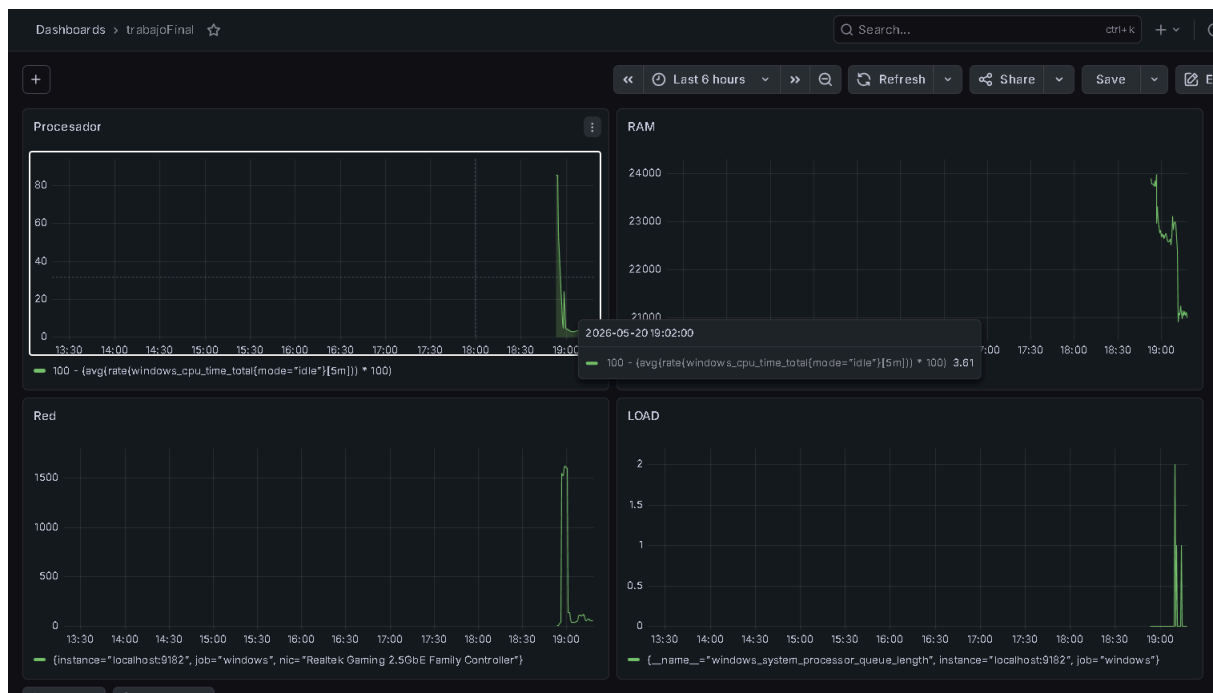
8.4. Tabla de Resultados del Monitoreo Grafana

Escenario de Prueba	CPU pico (%)	RAM usada pico (MB)	Carga sistema (load1)	Trafico red (KB/s)	
Juego inactivo (baseline)	5.05%	21149	1	118	

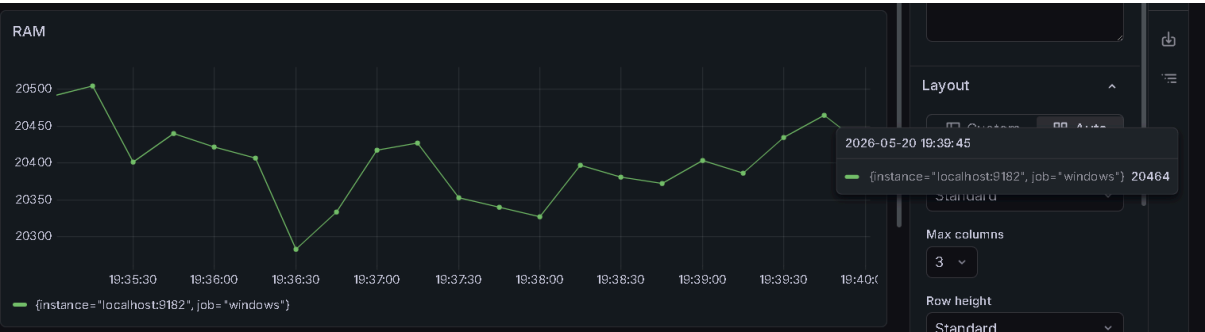
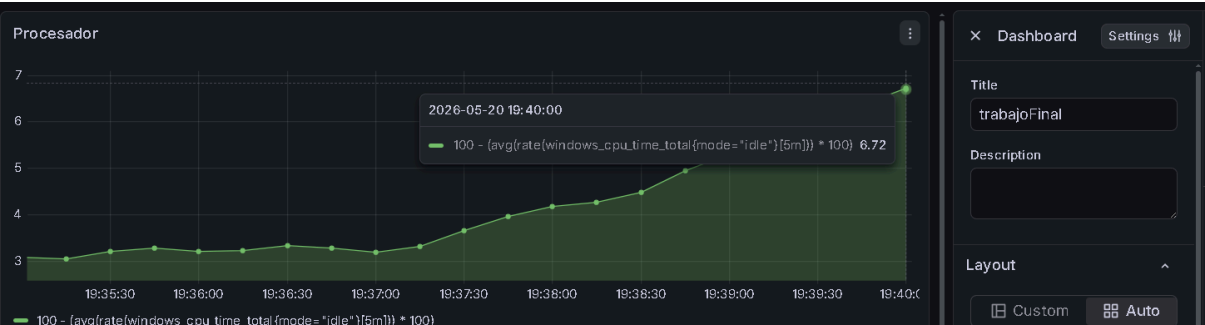
Juego con 500 bloques - Modo Normal	6.72%	20464	0	60.1	
Juego con 1000 bloques - Modo Normal	9.22%	20286	0	46.2	
Juego con 1000 bloques - Modo Paralelo	9.50%	20185	0	39.3	
JMeter 20 usuarios concurrentes	8.06%	20070	1	18.8	
JMeter 60 usuarios concurrentes	9.78%	20005	0	45	

8.5. Evidencias del Monitoreo

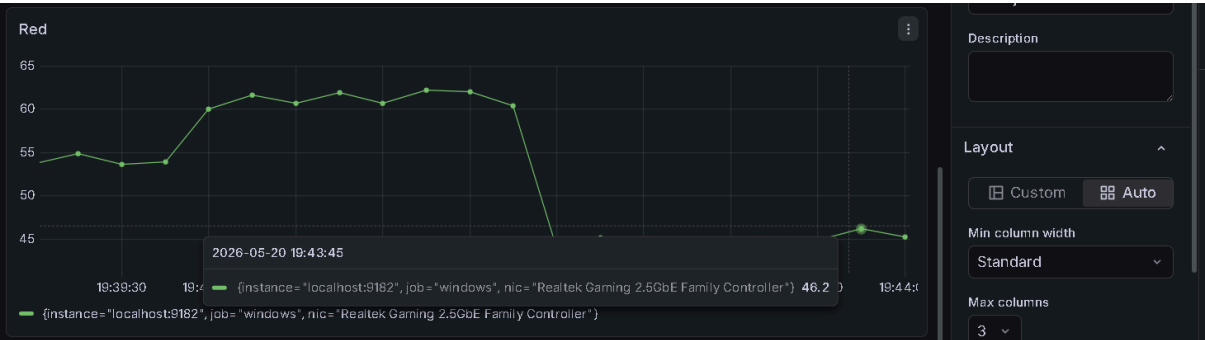
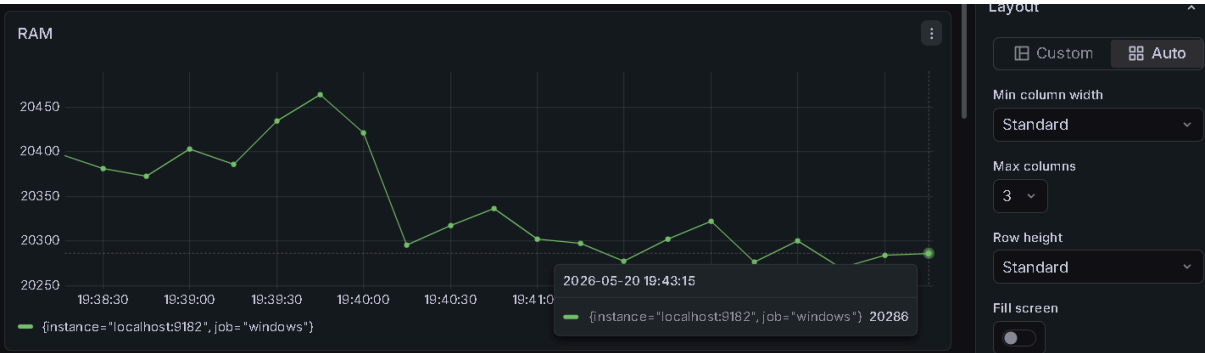
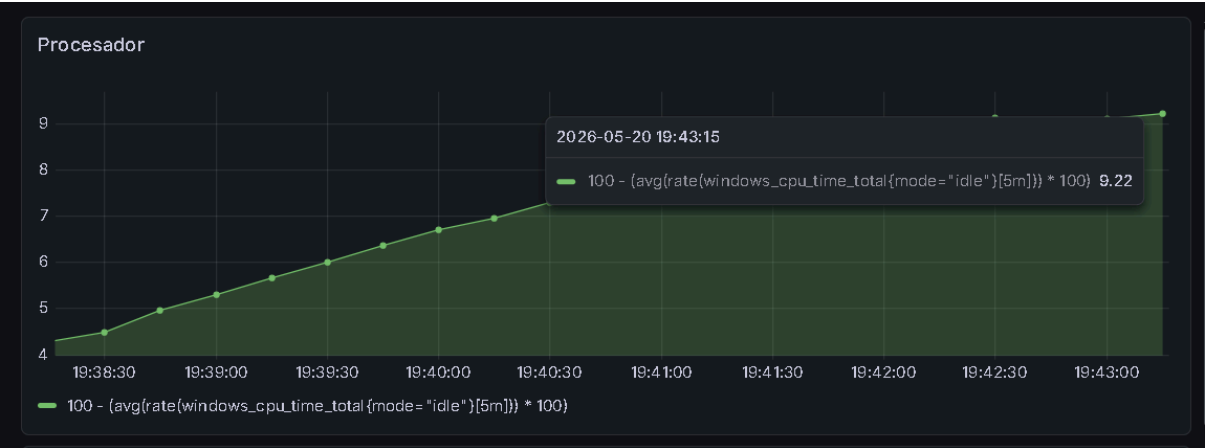
Juego Inactivo



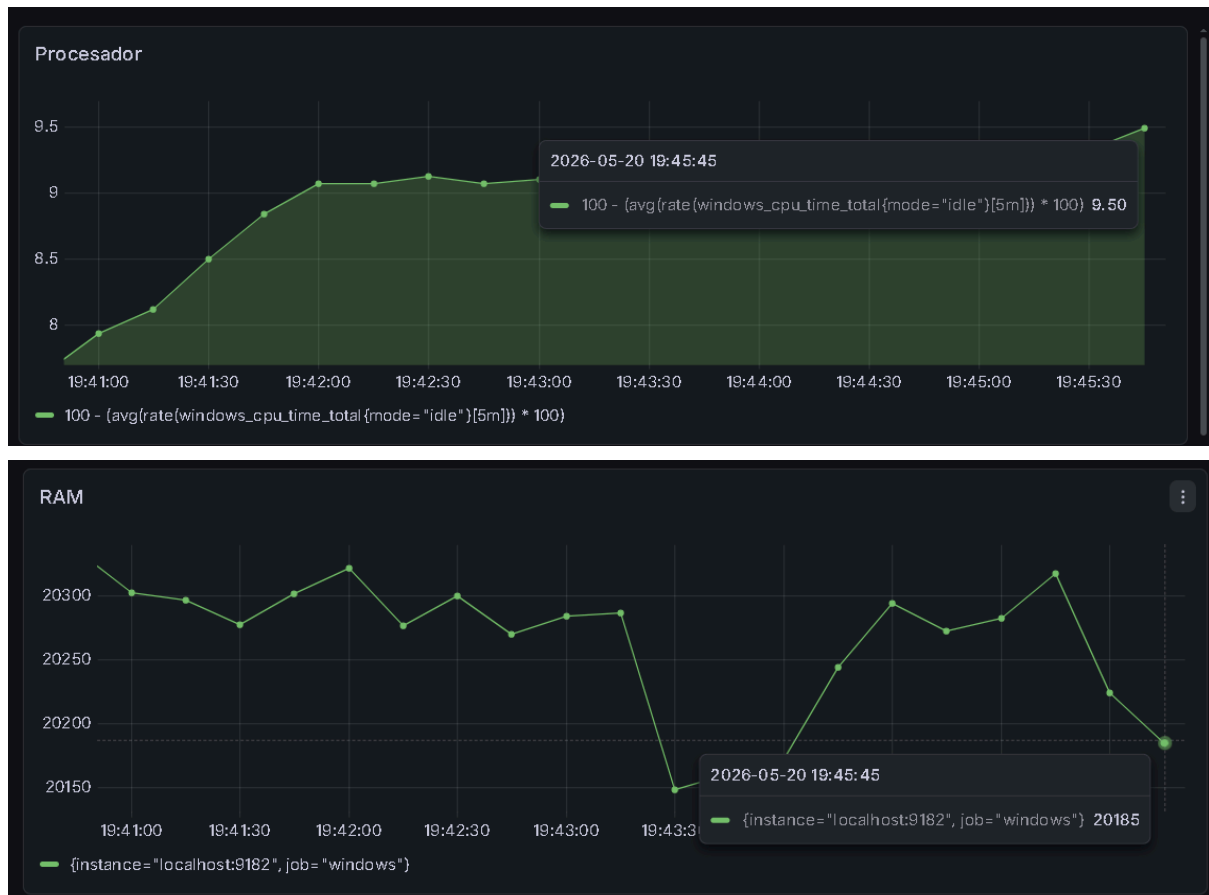
Juego con 500 Bloques



Juego con 1000 bloques

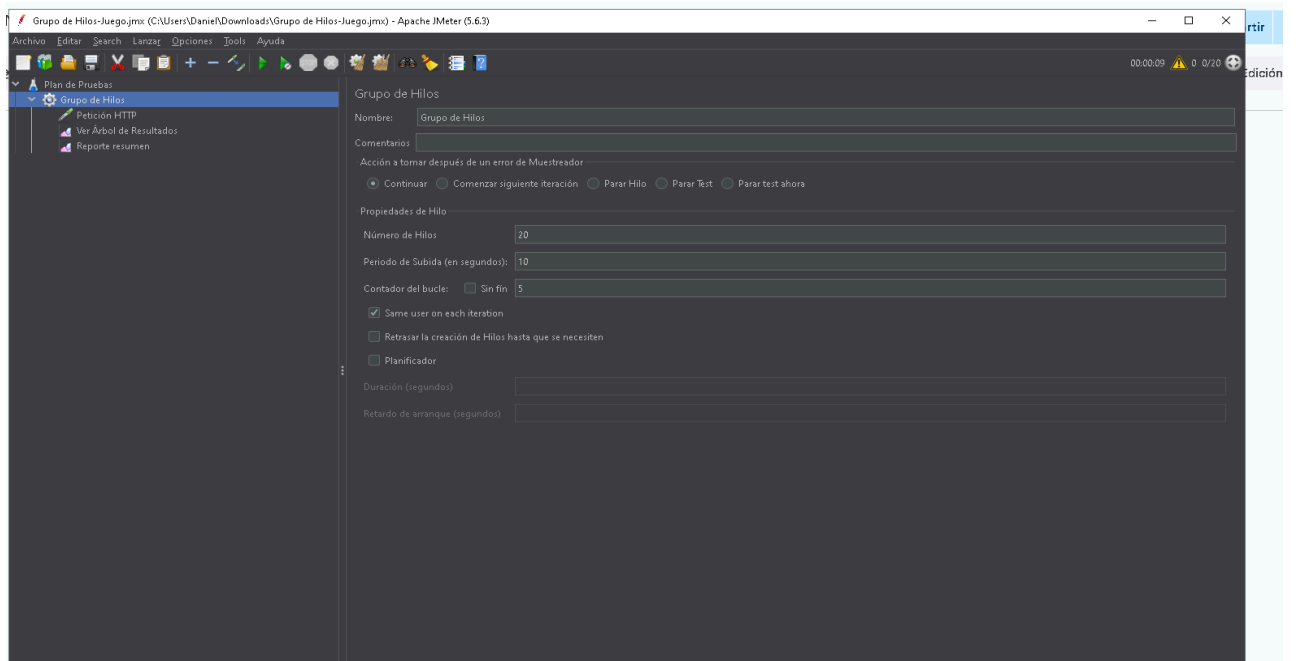


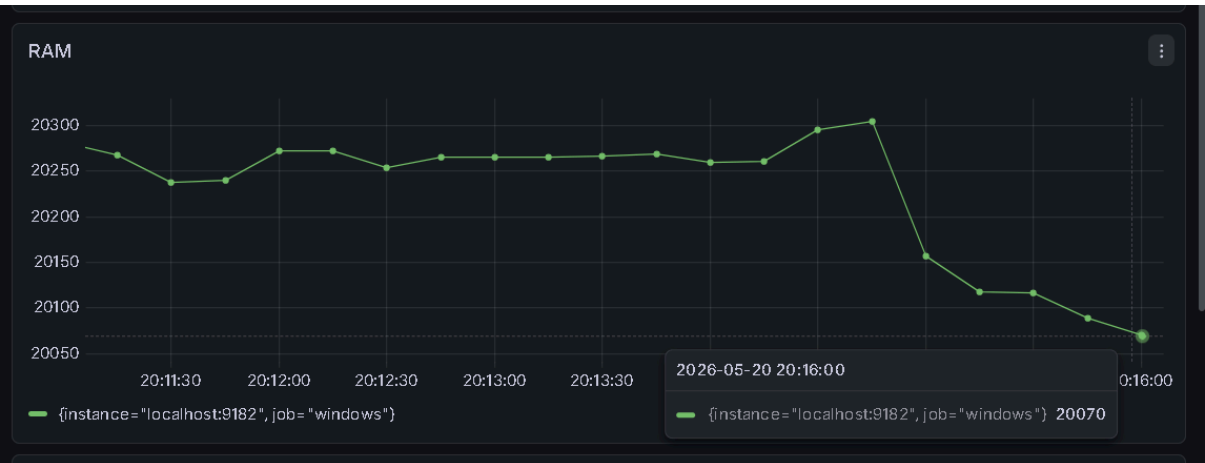
Juego con 1000 bloques en paralelo



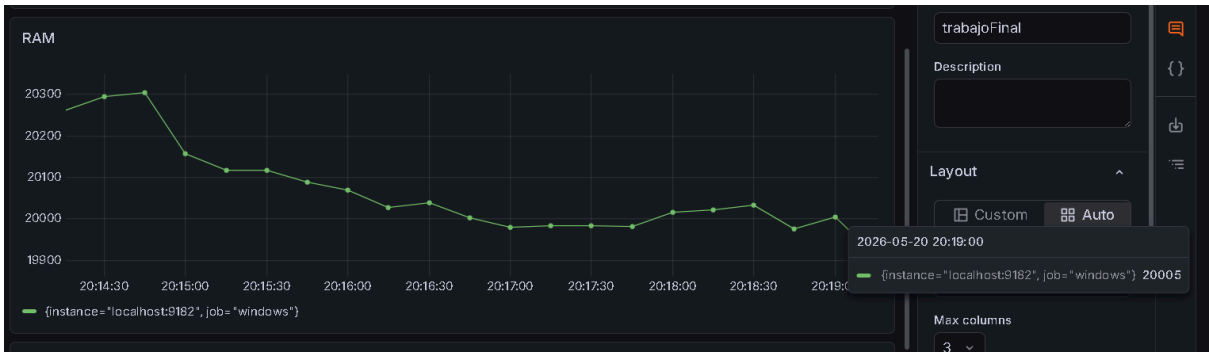
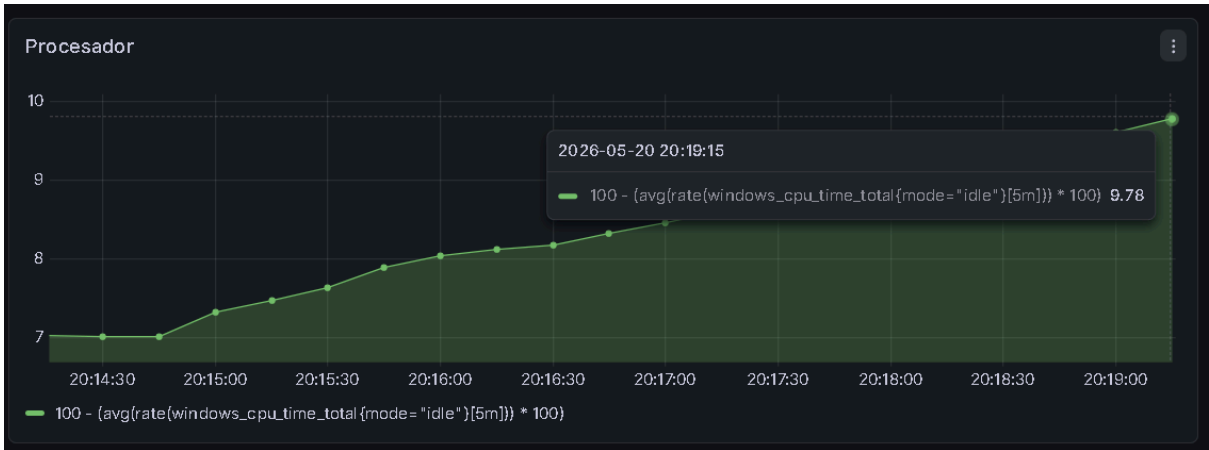
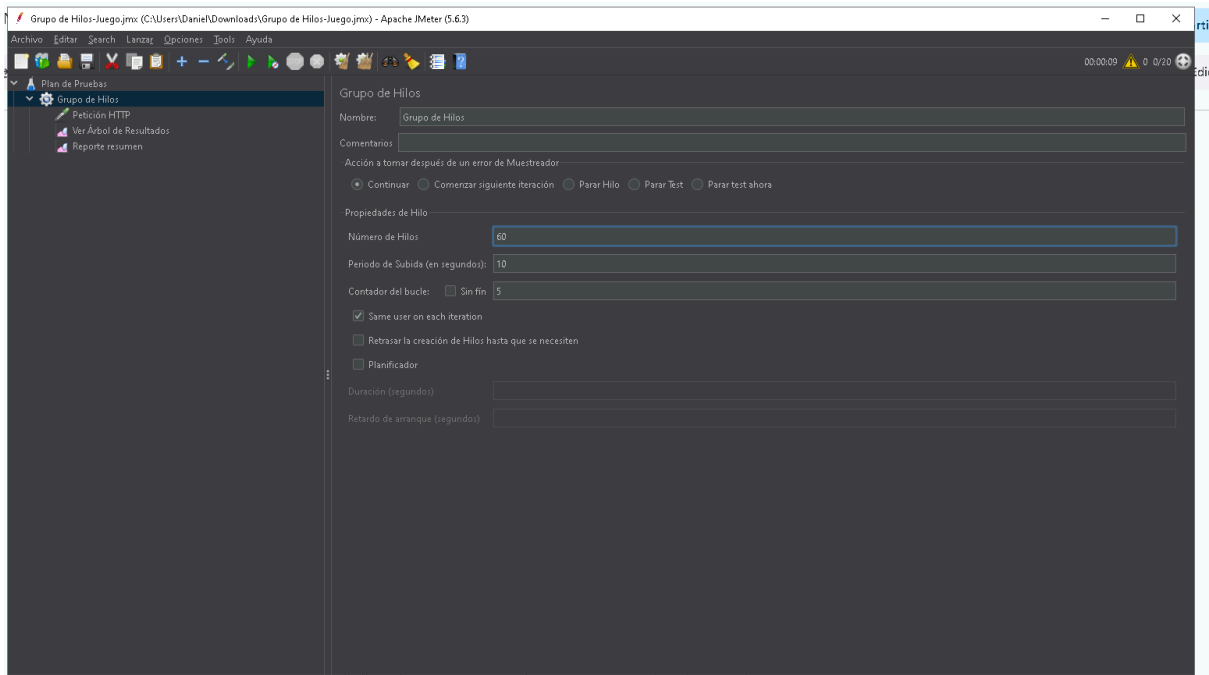


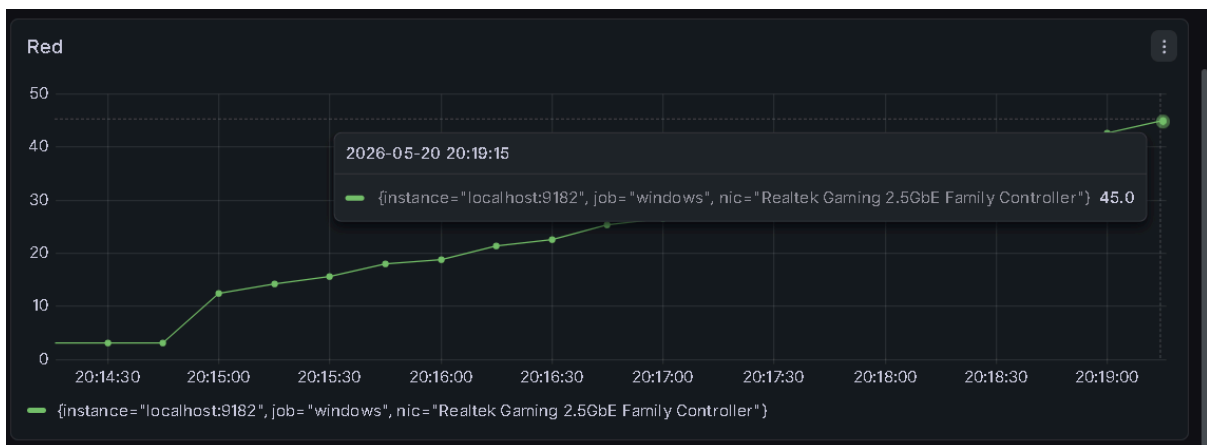
Juego con 20 peticiones en Jmeter





Prueba con Jmeter 60 peticiones





Las pruebas de carga realizadas con Apache JMeter permitieron evaluar el comportamiento del sistema bajo diferentes cantidades de usuarios concurrentes. Durante el monitoreo con Prometheus y Grafana se observó un uso estable de los recursos del sistema, manteniendo bajos tiempos de respuesta y sin presentar errores durante las pruebas.

9. Propuestas de Mejora

Durante el desarrollo de este laboratorio se identificaron diferentes aspectos que podrían mejorarse para optimizar el rendimiento general del aplicativo y el uso de recursos del sistema. Una de las principales mejoras sería optimizar la cantidad de objetos renderizados en pantalla, ya que durante las pruebas se observó que al agregar muchos bloques el consumo de memoria RAM y CPU aumentaba considerablemente, generando caídas de FPS y pequeñas ralentizaciones. Para solucionar esto, sería útil implementar técnicas de reutilización de objetos dentro del entorno 3D, evitando crear y eliminar elementos constantemente.

También sería importante mejorar el manejo de memoria en JavaScript y Three.js, liberando correctamente geometrías, texturas y objetos que ya no se estén utilizando. Durante algunas pruebas se observó que parte de la memoria seguía ocupada incluso después de eliminar bloques, lo que demuestra la importancia de controlar adecuadamente el Garbage Collector y evitar fugas de memoria en aplicaciones interactivas.

10. Conclusiones

El desarrollo de este laboratorio permitió comprender de manera más práctica cómo la arquitectura computacional influye directamente en el rendimiento de aplicaciones modernas desarrolladas para la web. A través de las pruebas realizadas se pudo observar que los computadores con arquitectura CISC ofrecieron un mejor desempeño en escenarios de alta carga gráfica, manteniendo una mayor estabilidad de FPS y tiempos de respuesta más rápidos en comparación con los dispositivos móviles con arquitectura RISC. Sin embargo, los dispositivos móviles demostraron una mejor eficiencia energética y un menor consumo de recursos en determinadas situaciones.